MySQL 与 MongoDB 日志读写性能对比

温子健

摘　要：本文基于统一日志模型和 REST 接口，比较 Spring Data JPA/MySQL 与 Spring Data MongoDB/MongoDB 的日志读写性能。实验使用四台云主机分两轮部署，预置 10 万条日志，在 1 至 200 线程下测试查询与单条写入，每档重复 3 次。200 线程时，两库查询吞吐量分别为 1,258.0、1,167.8 次/秒，写入吞吐量分别为 4,608.5、4,236.2 次/秒。72 次正式测量共记录 20,730,914 次请求、0 次错误。结合响应时间分布与资源监测，分析增加并发的吞吐收益和延迟变化。

关键词：MySQL；MongoDB；Spring Data JPA；JMeter；Prometheus

Log Read and Write Performance of MySQL and MongoDB

WEN Zi-Jian

Abstract: This experiment compares Spring Data JPA/MySQL and Spring Data MongoDB/MongoDB using a shared log model and REST API. Four cloud servers are reused in two sequential rounds. Each database starts with 100,000 records. Read and write workloads use 1 to 200 concurrent threads, with three repetitions per level. JMeter measures latency, throughput and errors, while Prometheus collects resource metrics every five seconds. At 200 threads, read throughput is 1258.0 requests/s for MySQL and 1167.8 requests/s for MongoDB; write throughput is 4608.5 and 4236.2 requests/s, respectively. The analysis relates latency distributions to application, database and load-generator resource usage.

Key words: MySQL; MongoDB; Spring Data JPA; JMeter; Prometheus

1　问题描述

日志服务需要持续写入事件，并按日志级别和时间范围查询历史记录。本文比较关系型表与文档集合在同一 Java 日志业务中的读写表现，重点考察并发增加时吞吐量、响应时间分布及资源占用的变化。两条链路使用相同接口、字段、请求生成规则和 JMeter 断言，使对照结果对应实际应用访问过程。

2　实验环境与方法

2.1　四机两轮部署

实验采用四台 Ubuntu 18.04 主机。第一轮运行 MySQL 和 JPA 应用，第二轮停止 MySQL 并切换为 MongoDB 及对应应用。两轮复用相同的数据库机、应用机、内网路径和资源配置，Compiler 的编译工作在压测开始前完成。

表 1　服务器配置与分工

主机	CPU 与可见内存	实验角色
Compiler	2 核，792 MiB	Java 17 编译与 JMeter 压测
Node1	2 核，1800 MiB	MySQL 或 MongoDB，分轮运行
Node2	2 核，792 MiB	当前被测 Spring 应用
Manager	2 核，1800 MiB	Prometheus 监控

表 2　软件与运行参数

项目	实际配置
应用	Spring Boot 3.3.13，Java 17.0.21，堆 256 MiB
编译与压测	Java 17.0.7，Maven 3.9.16，JMeter 5.6.3
MySQL	8.4.13，InnoDB 缓冲池 256 MiB
MongoDB	7.0.16，WiredTiger 缓存 0.25 GiB
数据库资源	两轮数据库容器内存上限均为 1 GiB
连接池	两条链路最大 20 个连接，最小 5 个
监控	Prometheus 3.7.3，node_exporter 1.10.2，5 秒采样

2.2　日志模型与统一接口

日志字段包括 id、traceId、level、source、message、clientIp 和 createdAt。id 采用 26 位 ULID，createdAt 统一保存 UTC 毫秒时间，message 最长 4096 字符。MySQL 以 CHAR(26)保存主键、DATETIME(6)保存时间；MongoDB 将 id 映射为字符串_id，以 BSON Date 保存时间。

POST /api/logs 执行单条写入并返回 HTTP 201 及保存的对象。GET /api/logs 接收 level、from、to、page、size，按左闭右开时间范围查询，返回 content、page、size、totalElements 和 totalPages。两套实现均按 createdAt 和 id 降序排列，并建立级别、时间、主键联合索引。

JPA 实现使用 EntityManager.persist 写入、JpaRepository 条件分页查询；MongoDB 实现使用 Spring Data MongoTemplate 执行 insert、count 和 find。两条读取链路均返回当前页与匹配总数。MongoTemplate 默认写入_class 类型元数据，HTTP 业务字段保持统一。MySQL 每次事务提交刷盘并关闭二进制日志，MongoDB 采用单机 WiredTiger 及 w=1、journal=true 写入确认。

3　实验设计

3.1　基准数据与请求

生成器以 20261111 为随机种子创建 10 万条日志，时间均匀覆盖 2026 年 11 月 1 日 UTC 全天，包含五种级别和八个服务来源。两库导入同一批业务字段，导入后核对记录数。固定查询为 INFO 级别、UTC 06:00 至 07:00、第一页 20 条，匹配总数为 2497 条。写入请求使用 250 字符正文和固定业务时间，服务端生成 ULID。

3.2　升压与重复测量

查询与写入分别采用 1、10、25、50、100、200 线程，每档重复 3 次，两种数据库共 72 次正式测量。每次运行前恢复 10 万条基准数据，随后记录 10 秒升压、30 秒预热和 120 秒正式测量。正式窗口按请求开始时间截取；窗口结束附近的收尾请求单独保留，原始 HTTP 采样行可完整复核。预热写入计入本次数据库增长，下一次运行前恢复基准。写入测试将全程成功请求数与数据库记录增长量核对，查询测试核对记录总数保持不变。

3.3　指标与监控

JMeter 使用非 GUI 模式，保存完整 HTTP 响应时间及成功状态，不扣除网络连接耗时。统计每次运行的吞吐量、错误率、平均值和 P50/P95/P99；各档汇总采用 3 次测量的中位数，误差线展示最小值至最大值。吞吐量按正式窗口请求数除以 120 秒计算。Prometheus 同步记录主机 CPU、可用内存、磁盘读写、应用堆和 GC 指标，并监控压测机资源。

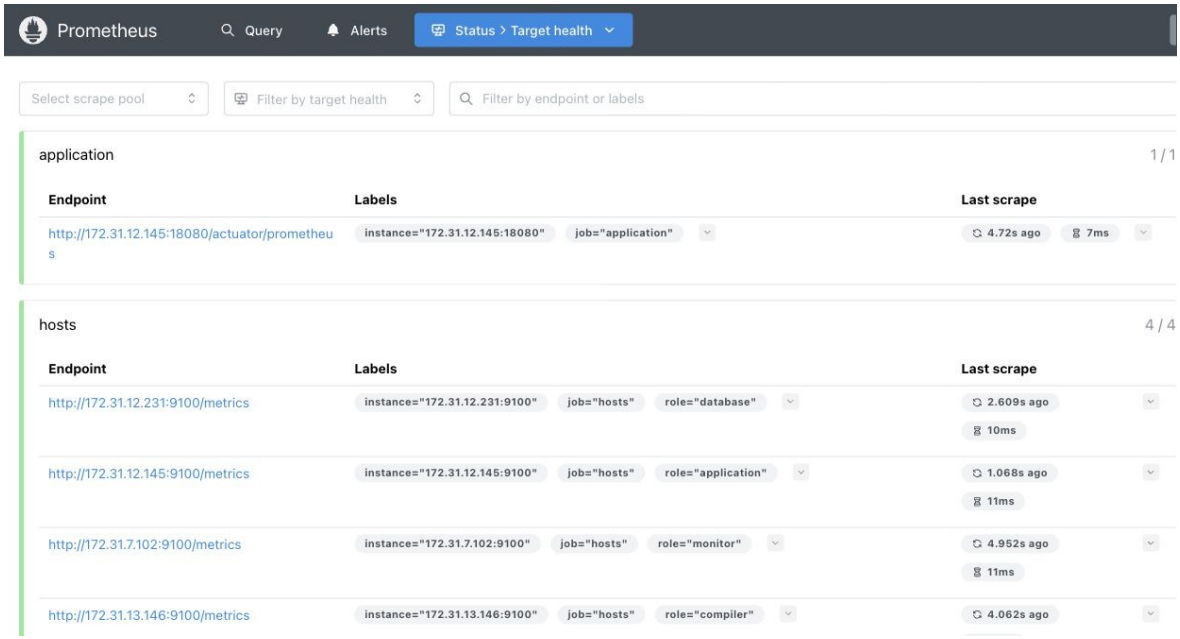


图 1　Prometheus 的应用与四台主机采集目标全部在线

3.4　功能与索引验收

两条实现分别通过 14 项 HTTP 检查，覆盖固定数据查询、分页互斥、空结果、负页码、页大小、非法级别、正文超长、UTC 毫秒精度及时间区间边界。固定查询的主键顺序与生成器计算的预期值一致。MySQL 的 EXPLAIN 和 MongoDB 的执行计划保存在 evidence 目录，用于核对联合索引的使用。



MYSQL 日志接口验收记录

实际HTTP调用输出。完整响应与14项检查见 mysql-smoke.json。

POST /api/logs　HTTP 201

```
{
  "id": "01M740K4TGRSPY3SKS9W5G0RJ6",
  "traceId": "smoke",
  "level": "INFO",
  "source": "smoke",
  "message": "课程实验日志",
  "clientIp": "::1",
  "createdAt": "2026-11-03T00:00:00.123Z"
}
```

GET /api/logs　HTTP 200

```
{
  "page": 0,
  "size": 20,
  "totalElements": 2497,
  "totalPages": 125,
  "content": [
    {
      "id": "01M6A6FT000000000000000WFD",
      "traceId": "trace-00029165",
```

图 2　MySQL 实际 POST 与 GET 接口响应记录



MONGO 日志接口验收记录

实际HTTP调用输出。完整响应与14项检查见 mongo-smoke.json。

POST /api/logs　HTTP 201

```
{
  "id": "01M747ANGDPHFW8BZ2RBD1RDWH",
  "traceId": "smoke",
  "level": "INFO",
  "source": "smoke",
  "message": "课程实验日志",
  "clientIp": "::1",
  "createdAt": "2026-11-03T00:00:00.123Z"
}
```

GET /api/logs　HTTP 200

```
{
  "page": 0,
  "size": 20,
  "totalElements": 2497,
  "totalPages": 125,
  "content": [
    {
      "id": "01M6A6FT000000000000000WFD",
      "traceId": "trace-00029165",
```

图 3　MongoDB 实际 POST 与 GET 接口响应记录

3.5　数据库记录与索引

下图展示两轮压测结束后数据库中保留的记录总数及实际日志节选。记录总数与最后一次写入测试的成功请求数对应，正文节选用于核对字段落库情况。

MYSQL 压测结束后的实际数据库记录

最终记录总数及createdAt、id降序的3条实际记录。MySQL正文节选显示前80个字符。

```
Wed Nov 11 05:45:30 UTC 2026
final_log_count
765118
id      trace_id        level   source  message_excerpt client_ip       created_at
01M7472TWQF4AQ5K8V08BD1W92      write-200-665118        INFO    course-service-0        course benchmark payload. course benchmark payload. course
benchmark payload. co   10.0.0.1        2026-11-02 00:00:00.000000
01M7472TWPH805H55EF76SDQ87      write-200-665117        INFO    course-service-0        course benchmark payload. course benchmark payload. course
benchmark payload. co   10.0.0.1        2026-11-02 00:00:00.000000
01M7472TWM2JMW50N9JYQWT0TP      write-200-665116        INFO    course-service-0        course benchmark payload. course benchmark payload. course
benchmark payload. co   10.0.0.1        2026-11-02 00:00:00.000000
```

图 4　MySQL 实际存储记录与最终记录总数

MONGO 压测结束后的实际数据库记录

最终记录总数及createdAt、id降序的3条实际记录。MySQL正文节选显示前80个字符。

```
Wed Nov 11 05:45:23 UTC 2026
Final log count: 766808
[
  {
    _id: '01M74J44XAT1AR3PM93ATKEG85',
    traceId: 'write-200-666808',
    level: 'INFO',
    source: 'course-service-0',
    message: 'course benchmark payload. course benchmark payload. course benchmark payload. course benchmark payload. course benchmark payload. cou
benchmark payload. course benchmark payload. course benchmark payload. course benchmark payload. course benchmark payload. ',
    clientIp: '10.0.0.1',
    createdAt: ISODate('2026-11-02T00:00:00.000Z'),
    _class: 'edu.lab4.LogEntry'
  },
  {
    _id: '01M74J44X8BDC5EBD8QNTVMWMZ',
    traceId: 'write-200-666807',
    level: 'INFO',
    source: 'course-service-0',
    message: 'course benchmark payload. course benchmark payload. course benchmark payload. course benchmark payload. course benchmark payload. cou
benchmark payload. course benchmark payload. course benchmark payload. course benchmark payload. course benchmark payload. ',
    clientIp: '10.0.0.1',
    createdAt: ISODate('2026-11-02T00:00:00.000Z'),
    _class: 'edu.lab4.LogEntry'
  },
```

图 5　MongoDB 实际存储记录与最终记录总数

联合索引与执行计划

MySQL 联合索引列为 level、created_at、id；MongoDB 联合索引列为 level、createdAt、_id。两条读取链路按相同条件和排序返回数据，执行计划原文保存在 evidence 目录。

MYSQL 索引与查询计划

```
VERSION()
8.4.13
Table   Non_unique      Key_name        Seq_in_index    Column_name     Collation       Cardinality     Sub_part        Packed  Null    Index_type
Comment Index_comment   Visible Expression
logs    0       PRIMARY 1       id      A       97470   NULL    NULL            BTREE                   YES     NULL
logs    1       idx_level_time_id       1       level   A       4       NULL    NULL            BTREE                   YES     NULL
logs    1       idx_level_time_id       2       created_at      D       97470   NULL    NULL            BTREE                   YES     NULL
logs    1       idx_level_time_id       3       id      D       93558   NULL    NULL            BTREE                   YES     NULL
@@innodb_buffer_pool_size       @@innodb_flush_log_at_trx_commit        @@log_bin
268435456       1       0
id      select_type     table   partitions      type    possible_keys   key     key_len ref     rows    filtered        Extra
1       SIMPLE  logs    NULL    range   idx_level_time_id       idx_level_time_id       50      NULL    2497    100.00  Using index condition
```

图 6　MySQL 联合索引和查询计划记录

MONGO 索引与查询计划

```
7.0.16
[
  {
    v: 2,
    key: {
      _id: 1
    },
    name: '_id_'
  },
  {
    v: 2,
    key: {
      level: 1,
      createdAt: -1,
      _id: -1
    },
    name: 'idx_level_time_id'
  }
]
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'lab4.logs',
    indexFilterSet: false,
    parsedQuery: {
      '$and': [
        {
          level: {
            '$eq': 'INFO'
          }
        },
        {
          createdAt: {
            '$lt': ISODate('2026-11-01T07:00:00.000Z')
          }
        },
```

图 7　MongoDB 联合索引和查询计划记录

4　实验结果与分析

4.1　查询负载测试

表中吞吐量和延迟均为同档 3 次测量的中位数，完整平均值、P50/P95/P99 和各次结果保存在 results/aggregate.csv 与 all-runs.csv。200 线程时，MySQL 的吞吐量为 1,258.0 次/秒、P95 为 305.0 ms；MongoDB 的吞吐量为 1,167.8 次/秒、P95 为 258.0 ms。

表 3　查询性能汇总

线程	**MySQL** 次/秒	**P95** ms	**P99** ms	**MongoDB** 次/秒	**P95** ms	**P99** ms
1	373.3	3.0	4.0	419.3	3.0	3.0
10	1198.1	12.0	15.0	1248.8	13.0	16.0
25	1304.7	30.0	36.0	1271.2	32.0	38.0
50	1307.4	73.0	109.0	1270.1	57.0	67.0
100	1285.8	145.0	210.0	1249.7	122.0	149.0
200	1258.0	305.0	444.0	1167.8	258.0	328.0

本组 MySQL 正式请求错误数为 0，MongoDB 为 0。两种实现各档的 3 次测量错误率均为 0%。图中的误差线体现重复测量的波动，吞吐量曲线与 P95 曲线共同反映并发增加后的收益和延迟代价。

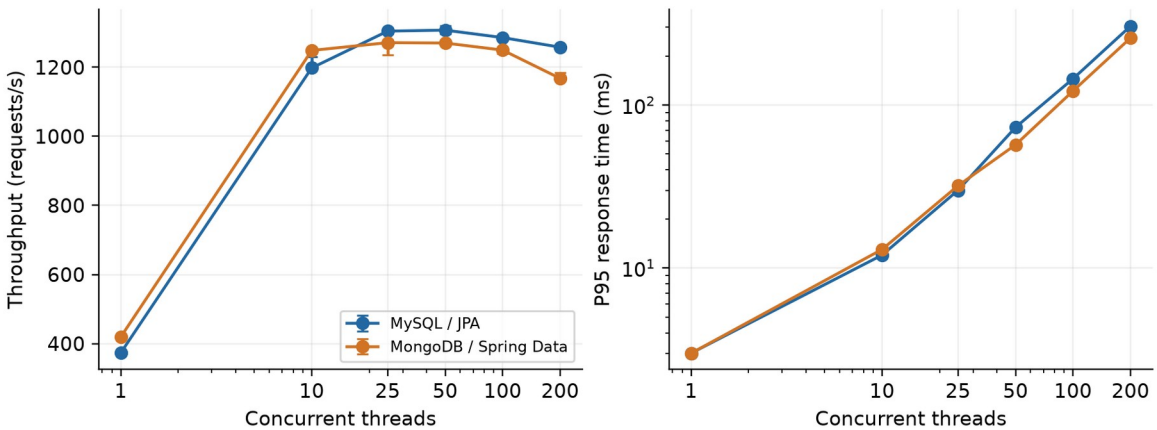


图 8　查询吞吐量及 P95 随并发变化，误差线为 3 次测量范围

在 200 线程档，MongoDB 相对 MySQL 的吞吐量低 7.2%、P95 低 15.4%。

在已测档位中，MySQL 的最高中位吞吐量为 1,307.4 次/秒，对应 50 线程、P95 为 73.0 ms；MongoDB 的最高中位吞吐量为 1,271.2 次/秒，对应 25 线程、P95 为 32.0 ms。

查询响应时间曲线

1 线程下，MySQL 三次测量的 P50/P95/P99 中位数为 3.0/3.0/4.0 ms，MongoDB 为 2.0/3.0/3.0 ms。下图展示第 1 次重复，趋势图按 5 秒窗口计算均值，百分位曲线使用该次正式窗口内全部响应时间。

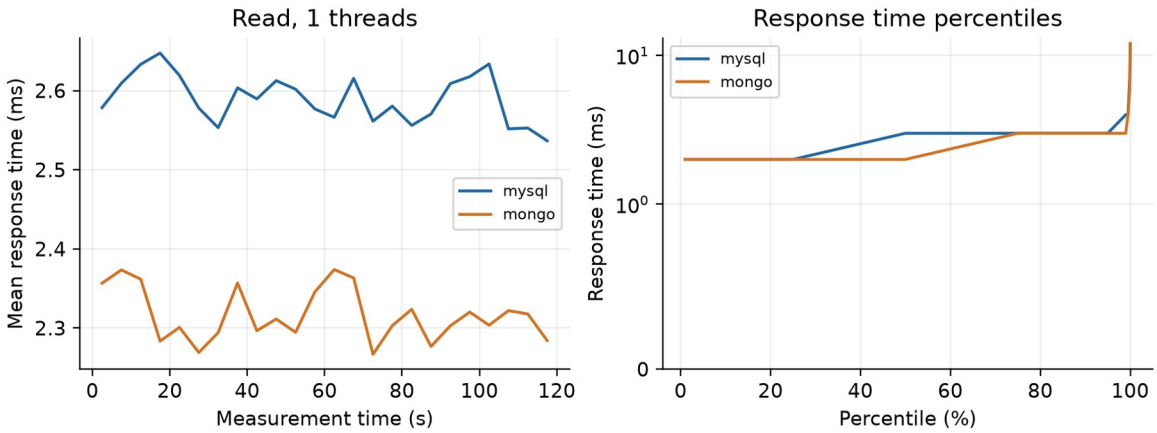


图 9　查询 1 线程第 1 次重复的响应时间趋势与百分位分布

25 线程下，MySQL 三次测量的 P50/P95/P99 中位数为 18.0/30.0/36.0 ms，MongoDB 为 19.0/32.0/38.0 ms。下图展示第 1 次重复，趋势图按 5 秒窗口计算均值，百分位曲线使用该次正式窗口内全部响应时间。

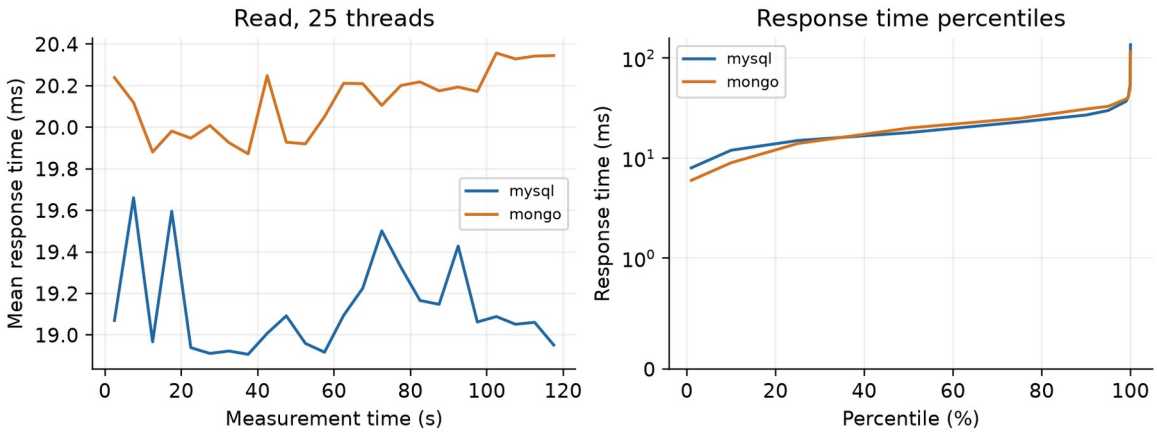


图 10　查询 25 线程第 1 次重复的响应时间趋势与百分位分布

查询高并发与资源占用

200 线程下，MySQL 三次测量的 P50/P95/P99 中位数为 158.0/305.0/444.0 ms，MongoDB 为 170.0/258.0/328.0 ms。下图展示第 1 次重复，趋势图按 5 秒窗口计算均值，百分位曲线使用该次正式窗口内全部响应时间。

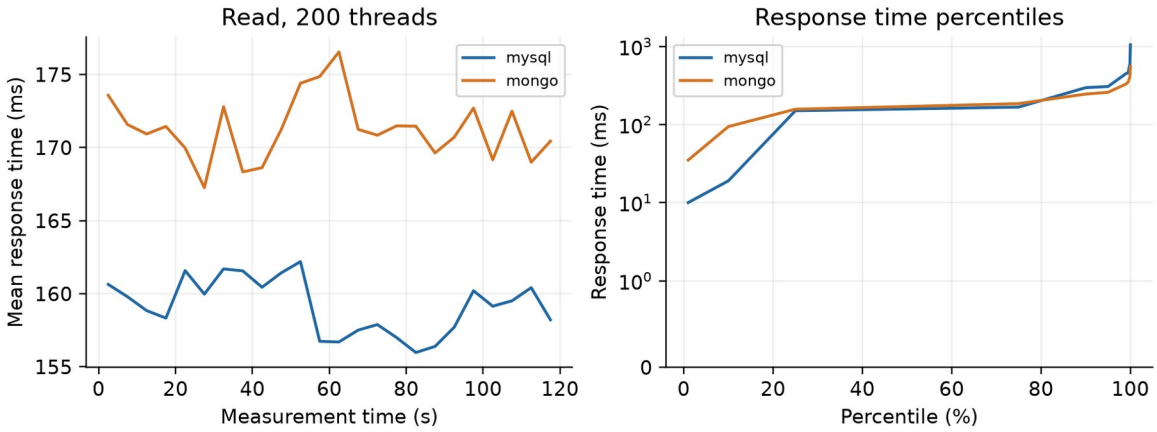


图 11　查询 200 线程第 1 次重复的响应时间趋势与百分位分布

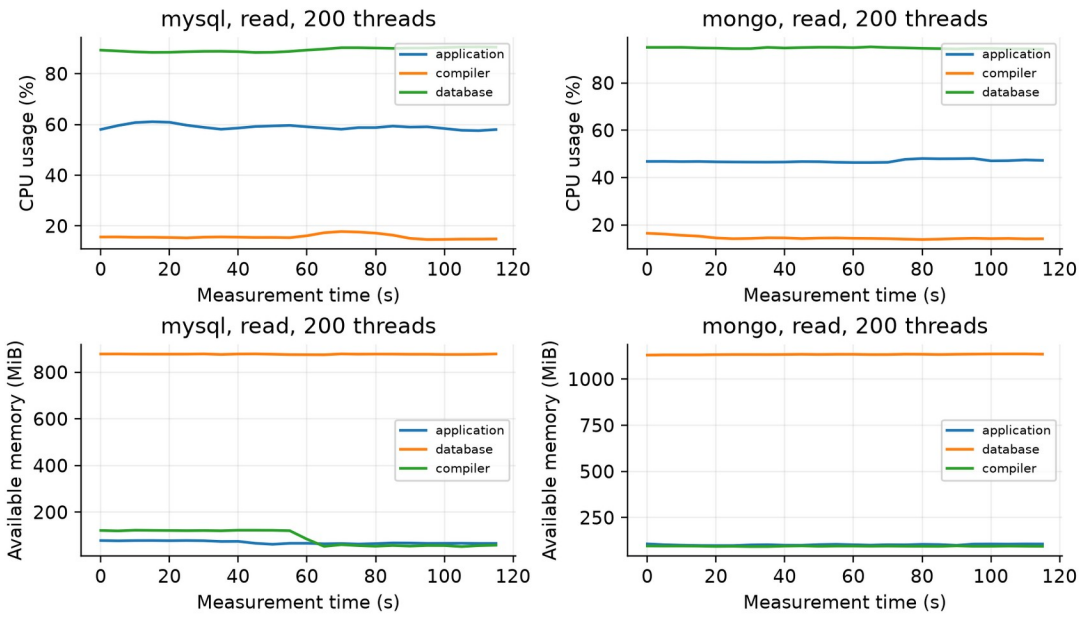


图 12　查询 200 线程第 1 次重复的主机 CPU 与可用内存

按压测机、应用机、数据库机顺序，三机平均 CPU 使用率为：MySQL 为 15.7%/59.0%/89.4%（压测机最小可用内存 52.4 MiB）；MongoDB 为 14.5%/47.0%/94.8%（压测机最小可用内存 92.8 MiB）。资源序列与延迟曲线采用同一正式窗口。

两组运行中 CPU 占用最高的环节分别为 MySQL 的数据库机（89.4%）、MongoDB 的数据库机（94.8%）。结合吞吐平台和 P95 增长，可以定位增加并发后资源压力集中的环节。

4.2　写入负载测试

表中吞吐量和延迟均为同档 3 次测量的中位数，完整平均值、P50/P95/P99 和各次结果保存在 results/aggregate.csv 与 all-runs.csv。200 线程时，MySQL 的吞吐量为 4,608.5 次/秒、P95 为 123.0 ms；MongoDB 的吞吐量为 4,236.2 次/秒、P95 为 94.0 ms。

表 4　写入性能汇总

线程	**MySQL** 次/秒	**P95** ms	**P99** ms	**MongoDB** 次/秒	**P95** ms	**P99** ms
1	608.6	2.0	3.0	689.3	3.0	3.0
10	3909.9	4.0	5.0	3214.2	4.0	6.0
25	4556.2	10.0	14.0	4612.8	8.0	11.0
50	5136.9	22.0	32.0	4475.3	18.0	23.0
100	4117.8	63.0	108.0	4342.3	44.0	61.0
200	4608.5	123.0	200.0	4236.2	94.0	135.0

本组 MySQL 正式请求错误数为 0，MongoDB 为 0。两种实现各档的 3 次测量错误率均为 0%。图中的误差线体现重复测量的波动，吞吐量曲线与 P95 曲线共同反映并发增加后的收益和延迟代价。

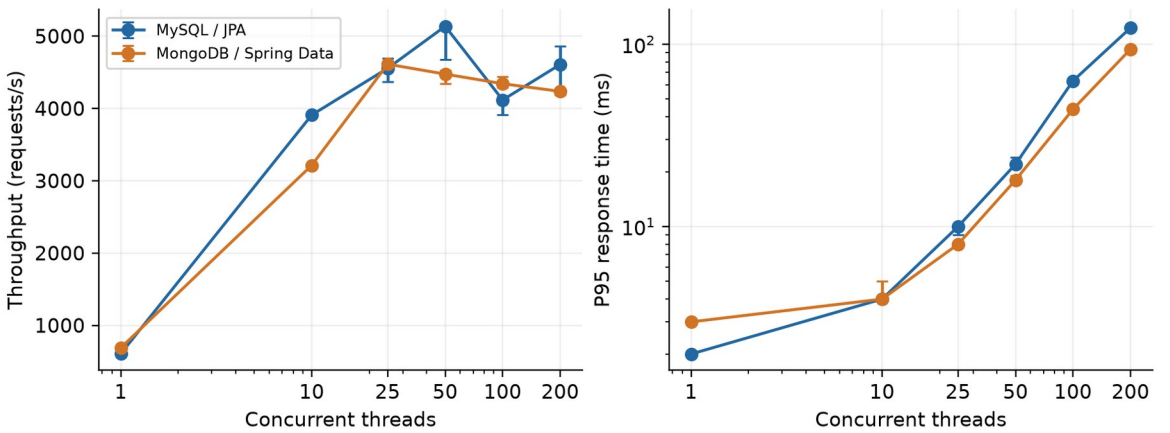


图 13　写入吞吐量及 P95 随并发变化，误差线为 3 次测量范围

在 200 线程档，MongoDB 相对 MySQL 的吞吐量低 8.1%、P95 低 23.6%。

在已测档位中，MySQL 的最高中位吞吐量为 5,136.9 次/秒，对应 50 线程、P95 为 22.0 ms；MongoDB 的最高中位吞吐量为 4,612.8 次/秒，对应 25 线程、P95 为 8.0 ms。

写入响应时间曲线

1 线程下，MySQL 三次测量的 P50/P95/P99 中位数为 2.0/2.0/3.0 ms，MongoDB 为 1.0/3.0/3.0 ms。下图展示第 1 次重复，趋势图按 5 秒窗口计算均值，百分位曲线使用该次正式窗口内全部响应时间。

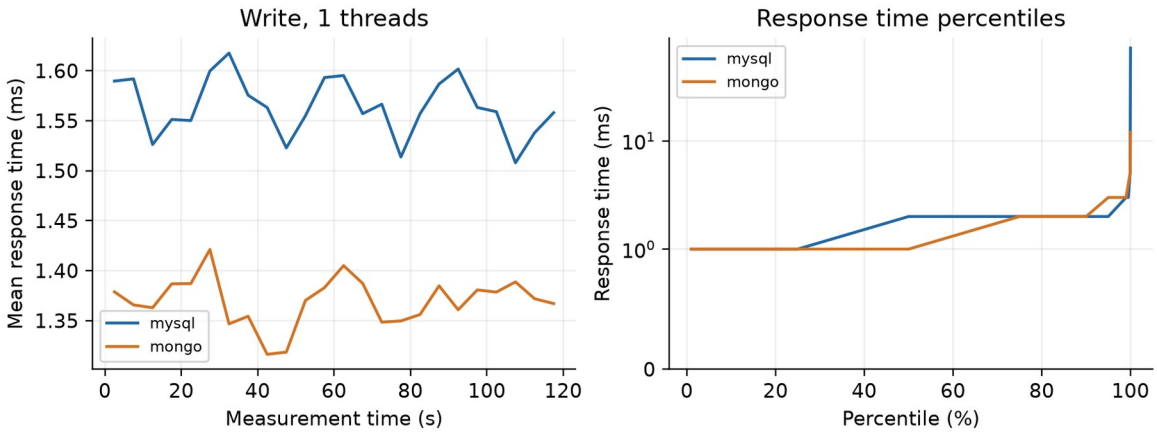


图 14　写入 1 线程第 1 次重复的响应时间趋势与百分位分布

50 线程下，MySQL 三次测量的 P50/P95/P99 中位数为 8.0/22.0/32.0 ms，MongoDB 为 11.0/18.0/23.0 ms。下图展示第 1 次重复，趋势图按 5 秒窗口计算均值，百分位曲线使用该次正式窗口内全部响应时间。

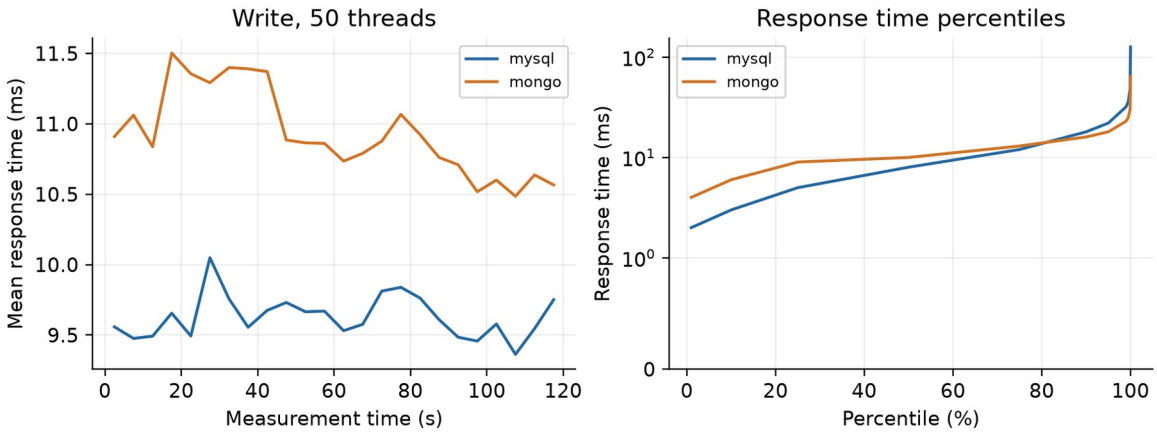


图 15　写入 50 线程第 1 次重复的响应时间趋势与百分位分布

写入高并发与资源占用

200 线程下，MySQL 三次测量的 P50/P95/P99 中位数为 33.0/123.0/200.0 ms，MongoDB 为 46.0/94.0/135.0 ms。下图展示第 1 次重复，趋势图按 5 秒窗口计算均值，百分位曲线使用该次正式窗口内全部响应时间。

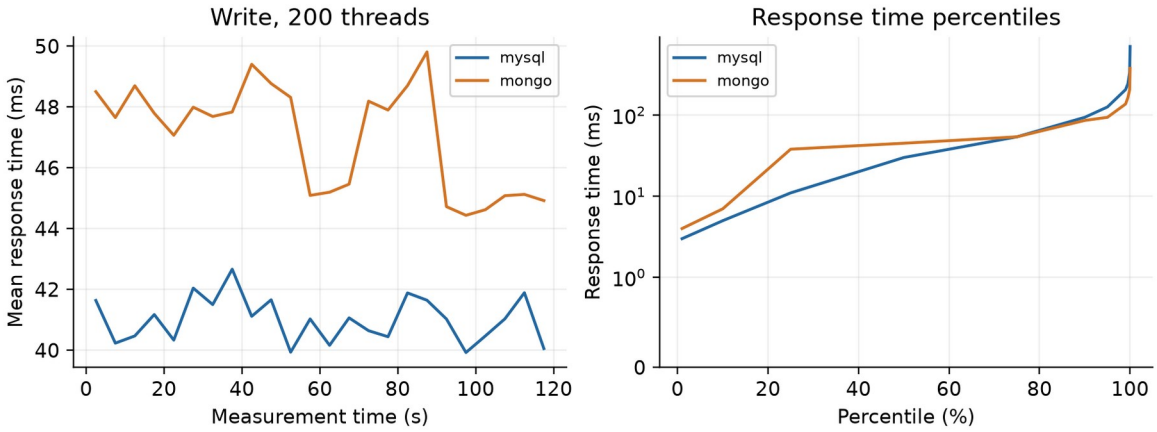


图 16　写入 200 线程第 1 次重复的响应时间趋势与百分位分布

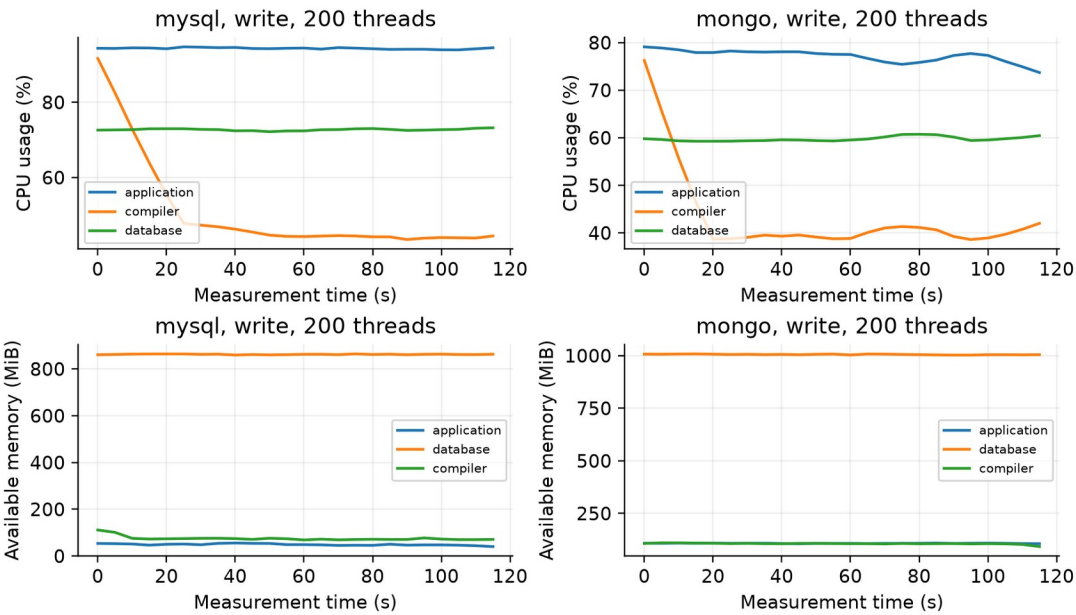


图 17　写入 200 线程第 1 次重复的主机 CPU 与可用内存

按压测机、应用机、数据库机顺序，三机平均 CPU 使用率为：MySQL 为 50.8%/94.3%/72.7%（压测机最小可用内存 68.9 MiB）；MongoDB 为 43.3%/77.2%/59.8%（压测机最小可用内存 92.4 MiB）。资源序列与延迟曲线采用同一正式窗口。

两组运行中 CPU 占用最高的环节分别为 MySQL 的应用机（94.3%）、MongoDB 的应用机（77.2%）。结合吞吐平台和 P95 增长，可以定位增加并发后资源压力集中的环节。

4.3　JMeter 原始报告页面

下图直接截取 200 线程第 1 次重复的 JMeter HTML 报告。每页保留同一实现的趋势与百分位曲线，完整页面原图保存在 screenshots 目录。

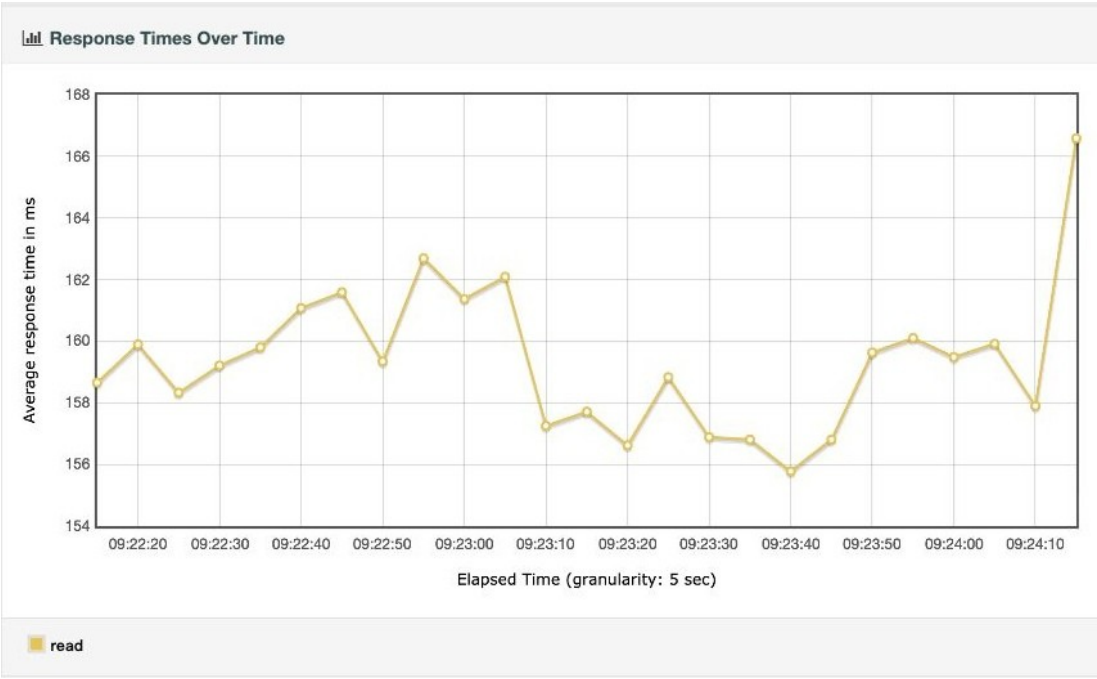


图 18　MySQL 查询 200 线程第 1 次重复的响应时间趋势

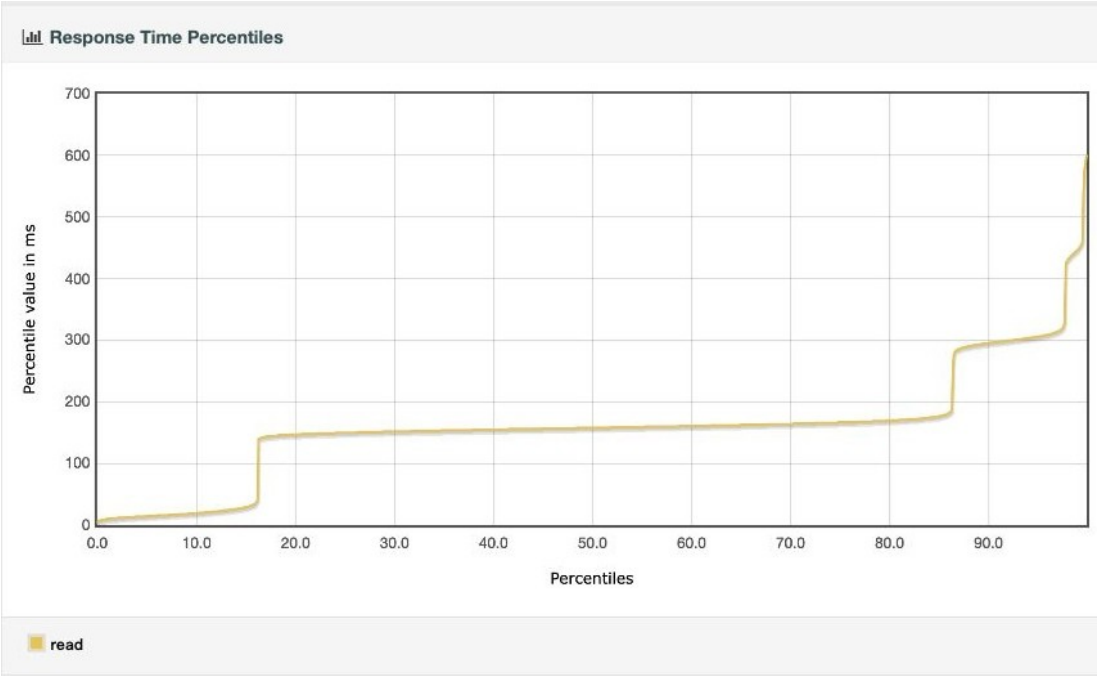


图 19　MySQL 查询 200 线程第 1 次重复的响应时间百分位

MongoDB 查询 JMeter 页面

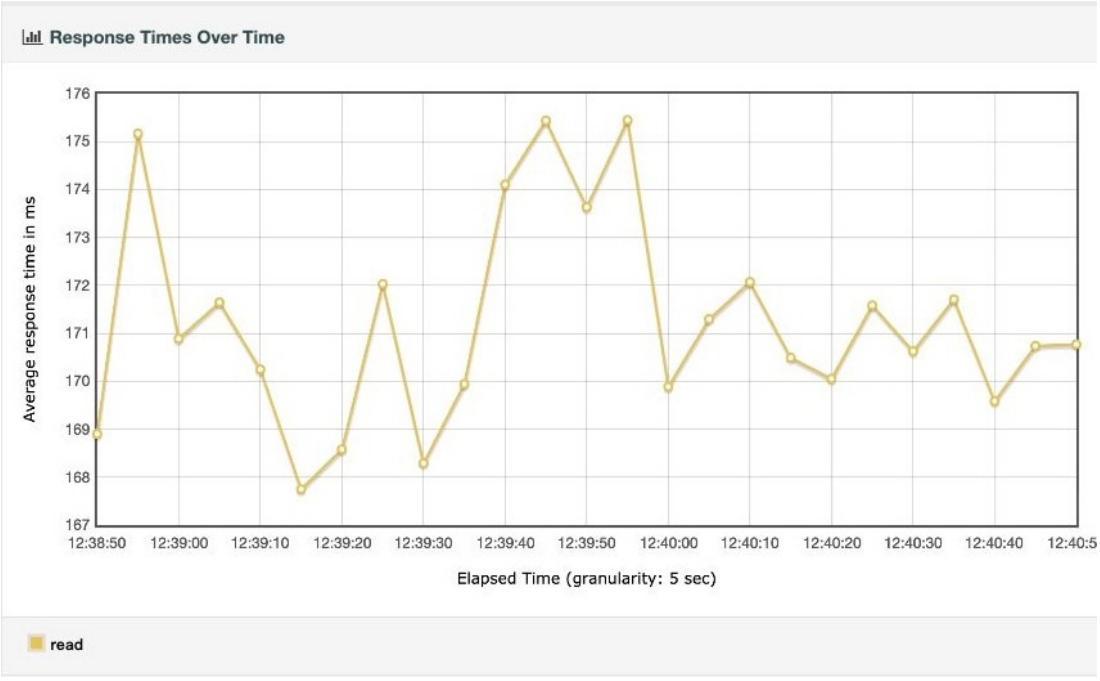


图 20　MongoDB 查询 200 线程第 1 次重复的响应时间趋势

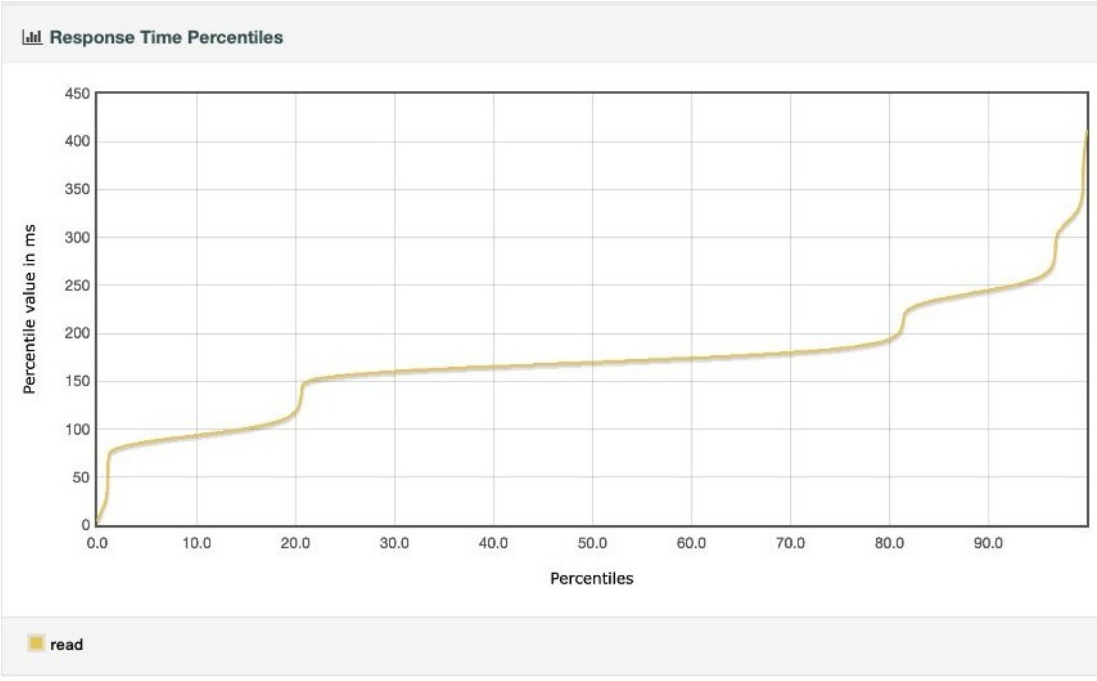


图 21　MongoDB 查询 200 线程第 1 次重复的响应时间百分位

MySQL 写入 JMeter 页面



图 22　MySQL 写入 200 线程第 1 次重复的响应时间趋势

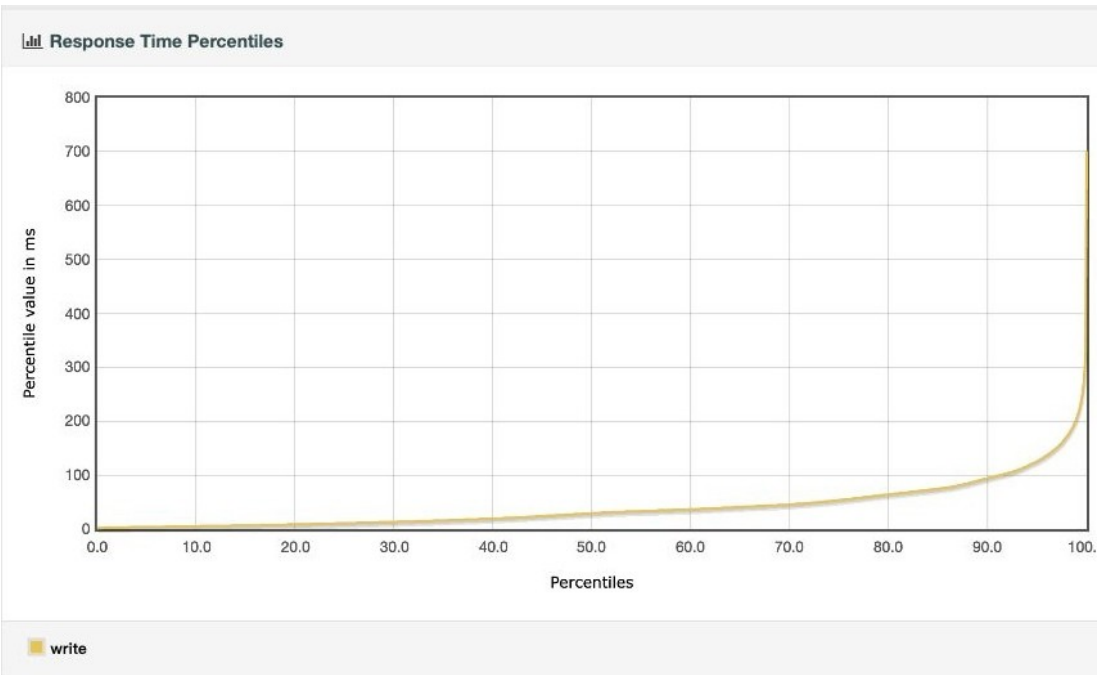


图 23　MySQL 写入 200 线程第 1 次重复的响应时间百分位

MongoDB 写入 JMeter 页面

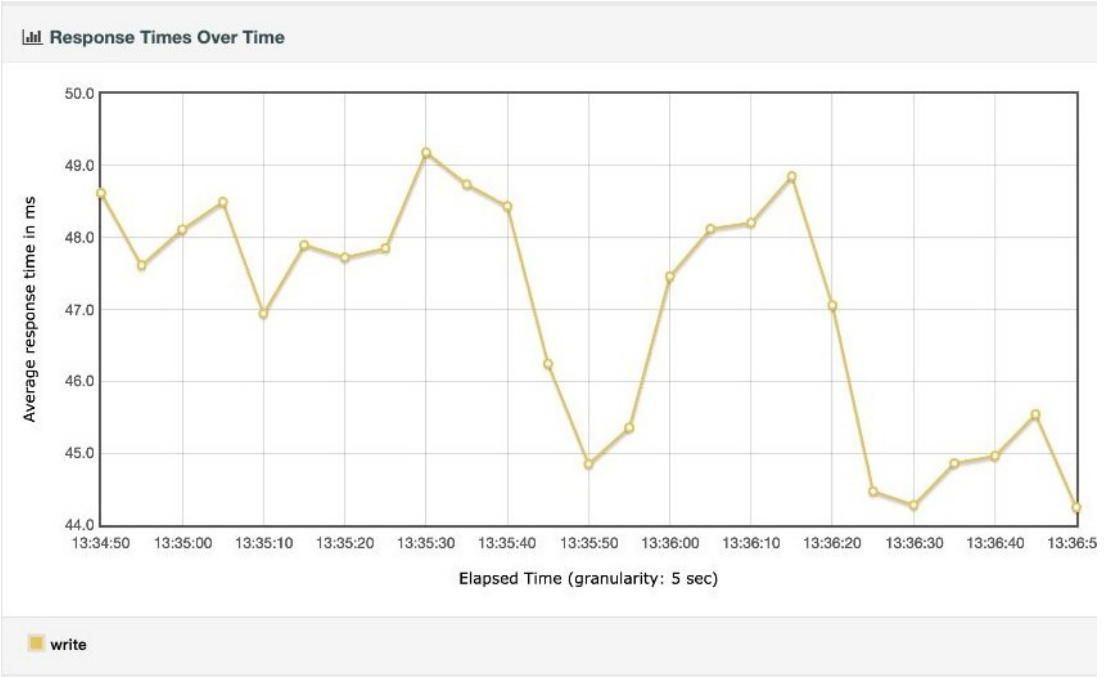


图 24　MongoDB 写入 200 线程第 1 次重复的响应时间趋势

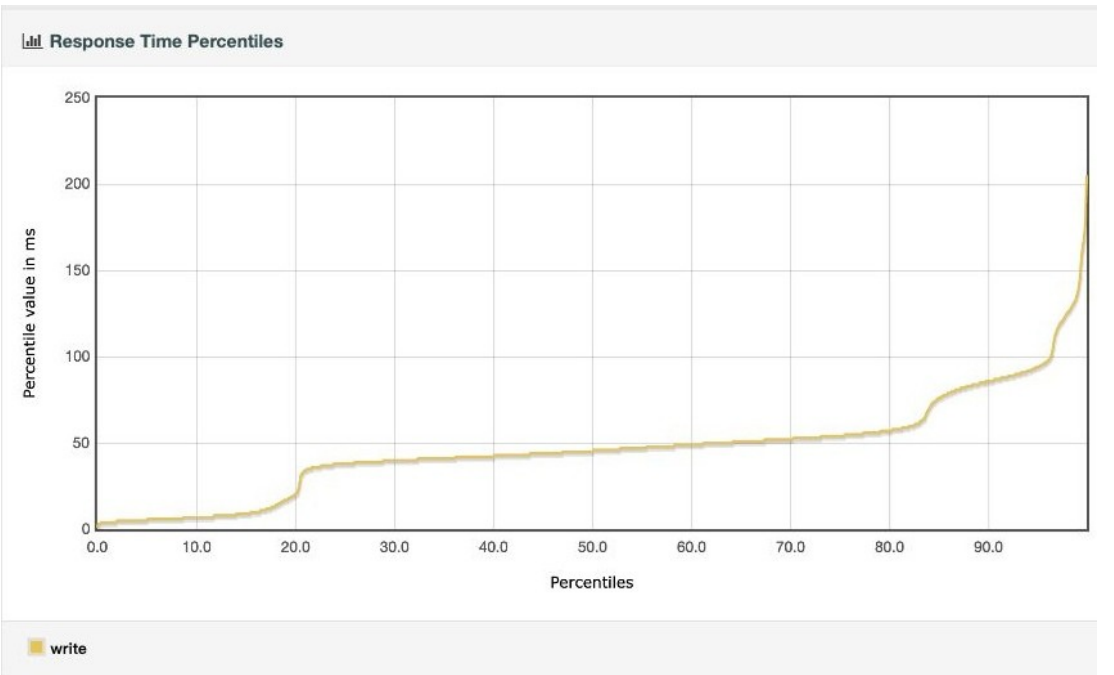


图 25　MongoDB 写入 200 线程第 1 次重复的响应时间百分位

5　总结

本实验完成了同一日志模型和接口下的 JPA/MySQL 与 Spring Data MongoDB/MongoDB 对照实现，并在四台云主机上通过两轮部署完成 72 次正式测量。200 线程时，MySQL 的吞吐量为 1,258.0 次/秒、P95 为 305.0 ms；MongoDB 的吞吐量为 1,167.8 次/秒、P95 为 258.0 ms。200 线程时，MySQL 的吞吐量为 4,608.5 次/秒、P95 为 123.0 ms；MongoDB 的吞吐量为 4,236.2 次/秒、P95 为 94.0 ms。这些结果描述了本实验硬件、查询选择率、索引、连接池和写入确认设置下的应用链路表现。

查询返回 20 条数据并计算 2497 条匹配记录的总数，写入采用单条持久化并等待确认；两类操作承担不同的数据库工作。并发增加后的性能变化应结合吞吐量、P95/P99 和主机资源判断。本文保留重复测量范围、原始错误和资源序列，使读写差异及性能转折均可追溯到具体运行。

表 5　实验完成与数据核验

核验项目	结果
正式测量	72 次，24 个实现与操作并发组合，各重复 3 次
正式请求数	20,730,914
正式错误数	0
记录数核对	全部运行均完成核对
原始数据	升压预热、正式窗口、收尾 JTL 无损压缩及 SHA-256
程序与材料	https://github.com/Baakarshan/mysql-mongodb-log-lab4

参考文献

[1] Spring. Spring Data JPA Reference Documentation. https://docs.spring.io/spring-data/jpa/reference/.

[2] Spring. Spring Data MongoDB Reference Documentation. https://docs.spring.io/spring-data/mongodb/reference/.

[3] Apache Software Foundation. Apache JMeter User Manual: Generating Dashboard Report. https://jmeter.apache.org/usermanual/generating-dashboard.html.

[4] MongoDB. Write Concern. https://www.mongodb.com/docs/v7.0/reference/write-concern/.

[5] Oracle. MySQL 8.4 Reference Manual: InnoDB Startup Options and System Variables. https://dev.mysql.com/doc/refman/8.4/en/innodb-parameters.html.